

Python Essentials – Evaluated Course Project

Prepared for VITyarthi Course Evaluation

Student:	Jay Patidar
Registration No.:	26BAI10449
Course:	Python Essentials
Project:	Student Record Management System

Academic Project Report

1. Introduction

The Student Record Management System is a Python-based academic project designed to manage student information and marks through a simple menu-driven interface. It demonstrates Python programming concepts such as classes, functions, lists/dictionaries, file handling, validation, modular programming, and basic testing.

2. Problem Statement

Manual handling of student records can make searching, updating marks, calculating results, and maintaining records difficult. The proposed system provides a simple digital workflow for adding, searching, updating, displaying, calculating, and deleting student records.

3. Objectives

- Create and maintain student records.
- Search students using ID or name.
- Update marks for four subjects.
- Calculate total, percentage, and pass/fail result.
- Display all stored students.
- Delete unwanted student records.
- Store data locally in JSON format.

4. Functional Requirements

- Add Student: accept student ID, name, course, and marks.
- Search Student: find a record by student ID or name.
- Update Marks: modify subject marks for an existing student.
- Calculate Result: calculate total, percentage, and result status.
- Display All Students: show all stored records.
- Delete Student: remove a selected student record.

5. Non-Functional Requirements

- Usability: menu-driven interaction should be simple to follow.
- Validation: marks and required fields should be checked.
- Reliability: records should be stored and loaded consistently.
- Maintainability: functionality is separated into modules.
- Error Handling: invalid input should not terminate normal program flow.
- Resource Efficiency: the system uses lightweight local JSON storage.

6. System Architecture

The project follows a modular architecture. The main interface is handled by `main.py`. Student data and calculations are represented by `student.py` and managed by `student_manager.py`. `storage.py` handles JSON persistence, `validators.py` performs input validation, and `reports.py` provides reporting-related functionality.

Module	Responsibility
<code>main.py</code>	Menu and user interaction
<code>student.py</code>	Student model and result calculations
<code>student_manager.py</code>	Add, search, update, display, delete operations
<code>storage.py</code>	Load/save JSON data
<code>validators.py</code>	Input validation
<code>reports.py</code>	Reporting utilities
<code>students.json</code>	Local student data storage

7. Workflow

- Start the program.
- Display the main menu.
- User selects an operation.
- System validates input and performs the operation.
- Student data is loaded from or saved to `students.json` where required.
- Result is displayed to the user.
- User can continue until Exit is selected.

8. Use Case Description

Primary actor: User. Main use cases: Add Student, Search Student, Update Marks, Calculate Result, Display All Students, Delete Student, and Exit.

9. Data Design

Each student record contains an ID, name, course, and marks for Python, Maths, English, and EVS. Records are persisted locally in JSON format so they can be loaded when the application starts.

10. Implementation

The project is implemented in Python using separate modules. The Student class provides result calculations. The manager module controls record operations. Validation functions check input values such as marks in the 0–100 range. JSON file handling provides simple persistent storage.

Main Menu

- 1. Add Student
- 2. Search Student
- 3. Update Marks
- 4. Calculate Total & Percentage
- 5. Display All Students
- 6. Delete Student
- 7. Exit

11. Testing

The implemented system was manually tested using a sample student record. The tested workflow included adding a student, searching by ID, updating marks, calculating total and percentage, displaying all students, deleting the student, and exiting the program.

Example test data: Student ID 101; sample name Jay; course B.Tech AI & ML; marks were entered and later updated. The core operations completed successfully during testing.

12. Design Decisions

- A menu-driven interface was selected because it is simple and directly demonstrates Python programming concepts.
- JSON was selected for lightweight local persistence without requiring an external database.
- Separate Python modules were used to improve organization and maintainability.
- Validation was separated from business logic to keep responsibilities clear.

13. Challenges

- Designing the record-management workflow.
- Keeping input validation separate from record operations.
- Maintaining data consistently in a JSON file.
- Testing multiple operations and edge cases.

14. Learnings

- Modular Python programming.
- Object-oriented programming using classes.
- File handling and JSON persistence.
- Input validation and error handling.

- Basic software architecture and documentation.
- Using GitHub for project version control.

15. Future Enhancements

- Add a graphical user interface using Tkinter.
- Add authentication and role-based access.
- Use a database such as SQLite.
- Add advanced reports and sorting/filtering.
- Add automated test coverage for more cases.

16. References

- Python documentation and course learning materials.
- VITyarthi project requirements and evaluation instructions.
- GitHub repository documentation.

Note: Screenshots of the running project can be added to the final submitted report to document the implementation and results.